Needless to say, when we have to compute thousands of routes per hour, using a more efficient search can save a lot of computation, allow shipments to leave the factory faster, and ultimately save money.

One again, we can get a theoretical guarantee about A* performance: if we fix the heuristic and distance, then for **any** consistent heuristic, A* will not just be optimal, but also optimally-efficient: this means that no other algorithm is guaranteed to expand fewer nodes than A*.

That said, the closer the estimate is to the actual distance of a vertex to the goal, the faster the algorithm will be in reaching the goal: as an example, try to come up with a better heuristic for the graph in figure 14.22 (hint: what could be more precise than straight-line distance?).

While we are happy to have a way to guarantee that A* will find the optimal solution, we also mentioned that sometimes it is acceptable to settle for a sub-optimal one: especially when the cost of traversal is high, or there are constraint on the response time, we might want to choose a non-admissible heuristic that however guarantees faster convergence..

14.5.2 Heuristics as a Way to Balance Real-time Data

That concludes our discussion about optimality of search; we had also mentioned there is at least another scenario where A* can prove itself particularly useful: let’s briefly explore it.